Cihan University — Erbil

Department of Informatics and Software Engineering

# CHATZY

## Real-Time Chat Application with AI Integration

*Final Year Project Report*

**Students:** Birhat Tofiq | Hiwa Nihmat | Ahmad Qasim

**Supervisor:** Ms. Duaa

**Department:** Informatics and Software Engineering

**Institution:** Cihan University — Erbil

**Academic Year:** 2025 – 2026

**Submission:** April 2026

# Declaration

We hereby declare that this project and the accompanying report are our own original work. All sources referenced and used in this report have been acknowledged. This work has not been submitted for any other academic qualification.

| Student | Signature | Date |
|---|---|---|
| Birhat Tofiq | _____ | _____ |
| Hiwa Nihmat | _____ | _____ |
| Ahmad Qasim | _____ | _____ |

## Final Year Project — Full Research & Technical Documentation

Students:

1. Birhat Tofiq

2. Hiwa Nihmat

3. Ahmad Qasim

Supervisor: Ms. Duaa

Department: Informatics and Software Engineering

Institution: Cihan University — Erbil

Academic Year: 2025 – 2026

Date of Submission: April 2026

## Declaration

We hereby declare that this project and the accompanying report are our own original work. All sources referenced and used in this report have been acknowledged. This work has not been submitted for any other academic qualification.

| Student | Signature | Date |
|---|---|---|
| Birhat Tofiq | _____ | _____ |
| Hiwa Nihmat | _____ | _____ |
| Ahmad Qasim | _____ | _____ |

## Acknowledgements

We would like to express our sincere gratitude to our supervisor, Ms. Duaa, for her continuous guidance, valuable feedback, and encouragement throughout the duration of this project. Her expertise and constructive suggestions played a significant role in shaping the direction and quality of this work.

We also thank the Department of Informatics and Software Engineering at Cihan University — Erbil for providing the academic environment, resources, and support necessary to complete this project.

Finally, we are grateful to our families and colleagues for their patience, moral support, and understanding during the development and writing phases of this project.

## Abstract

This document presents the design, development, and evaluation of CHATZY — a cross-platform, real-time mobile chat application built with the Flutter framework and powered by Google Firebase cloud services. The system delivers a modern, feature-rich instant messaging experience that incorporates real-time communication, AI-driven conversation assistance, ephemeral story sharing, group messaging, and an animated AI character driven by sentiment analysis.

CHATZY is designed following a layered software architecture that separates presentation, business logic, state management, and data access concerns. The backend leverages Firebase Authentication for identity management, Cloud Firestore for synchronized real-time data, and Firebase Cloud Storage for binary media assets. An AI module using Google Gemini 2.0 Flash provides intelligent chatbot functionality and sentiment-based character animation.

The application targets Android and web platforms and is built with a glass-morphism visual design language using Material 3 principles. The result is a production-quality messaging application comparable in feature scope to commercial platforms, demonstrating practical application of mobile development, cloud computing, and applied artificial intelligence.

Keywords: Flutter, Firebase, Firestore, Real-Time Chat, Dart, AI, Gemini, Glass UI, Provider, Cross-Platform

## List of Abbreviations

| Abbreviation | Full Term |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| APK | Android Package Kit |
| Auth | Authentication |
| BaaS | Backend-as-a-Service |
| DB | Database |
| E2E | End-to-End (Encryption) |
| FCM | Firebase Cloud Messaging |
| GLB | GL Transmission Format Binary (3D model file) |
| GPU | Graphics Processing Unit |
| HTML | HyperText Markup Language |
| HTTP/HTTPS | HyperText Transfer Protocol (Secure) |
| IDE | Integrated Development Environment |
| JSON | JavaScript Object Notation |
| LLM | Large Language Model |
| ML | Machine Learning |
| NoSQL | Non-Relational Database |
| OOP | Object-Oriented Programming |
| OS | Operating System |
| RAM | Random Access Memory |
| SDK | Software Development Kit |
| SQL | Structured Query Language |
| SQLite | Lightweight SQL Relational Database |
| UI | User Interface |
| UID | Unique Identifier |
| URL | Uniform Resource Locator |
| UX | User Experience |
| UUID | Universally Unique Identifier |

## Table of Contents

- Declaration

- Acknowledgements
- Abstract
- List of Abbreviations

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement

1.3 Aims and Objectives

1.4 Scope of the Project

1.5 Report Structure

2. Literature Review

2.1 Evolution of Instant Messaging Applications

2.2 Cross-Platform Mobile Development Frameworks

2.3 Backend-as-a-Service (BaaS) Platforms

2.4 Artificial Intelligence in Chat Applications

2.5 Ephemeral Content in Social Messaging

2.6 Summary of Related Work

3. Requirements Analysis

3.1 Functional Requirements

3.2 Non-Functional Requirements

3.3 System Requirements

3.4 Use Case Diagram

3.5 Use Case Descriptions

3.6 Application Flowchart

4. System Design and Architecture

4.1 High-Level Architecture

4.2 Technology Stack

4.3 Application Layer Architecture

4.4 Navigation Architecture

4.5 State Management Design

4.6 Firebase Architecture

4.7 AI Module Architecture

4.8 Application Screenshots

5. Database Design

5.1 Cloud Firestore Data Model
5.2 Firebase Storage Structure
5.3 Local SQLite Cache
5.4 Security Rules
6. Implementation
6.1 Development Environment
6.2 Project Structure
6.3 Authentication Module
6.4 Real-Time Messaging Module
6.5 AI Chatbot Module
6.6 Stories Module
6.7 User Interface and Theming
6.8 Media Handling
6.9 Offline Support
7. Testing and Evaluation
7.1 Testing Strategy
7.2 Functional Test Cases
7.3 Performance Considerations
7.4 Security Evaluation
8. Results and Discussion
8.1 Feature Completion Summary
8.2 Challenges and Solutions
8.3 Limitations
9. Conclusion and Future Work
9.1 Conclusion
9.2 Future Work
10. References
11. Appendices
A. Full Dependency List
B. Firestore Security Rules
C. Hardware and Software Requirements

# 1. Introduction

## 1.1 Background and Motivation

Instant messaging has become one of the most prevalent forms of digital communication worldwide. Applications such as WhatsApp, Telegram, and Discord process billions of messages daily, and users now expect richer, more intelligent experiences beyond simple text exchange. These expectations include real-time media sharing, ephemeral stories, AI-powered assistants, and highly polished visual interfaces.

The convergence of mature cross-platform mobile frameworks (such as Flutter), scalable cloud backends (such as Firebase), and accessible AI APIs (such as Google Gemini) has made it feasible for individual developers and small teams to build professional-grade messaging applications. This project takes advantage of these technologies to build CHATZY — a fully functional, production-quality chat application developed as a final year project.

The motivation for this project stems from the desire to apply theoretical knowledge of software engineering, mobile development, database design, and artificial intelligence in the context of a single cohesive and practical product.

## 1.2 Problem Statement

Existing open-source chat application templates are either oversimplified (lacking features such as group chat, presence, or AI) or too complex and poorly structured for educational purposes. There is a need for a well-architected reference implementation that demonstrates:

- Real-time bidirectional data synchronization.
- Secure user authentication and profile management.
- AI-enhanced user interaction through sentiment analysis and chatbot.
- Ephemeral content (stories) with automatic expiry.
- A modern, accessible, and aesthetically refined user interface.

## 1.3 Aims and Objectives

Aim: To design, develop, and evaluate a cross-platform real-time chat application with integrated AI features.

Objectives:

12. Implement secure user registration and authentication using Firebase Authentication.
13. Enable real-time private and group messaging with delivery and read receipts.
14. Integrate Google Gemini 2.0 Flash for an interactive AI chatbot and sentiment analysis.
15. Build an ephemeral stories feature with 24-hour auto-expiry.
16. Develop a glass-morphism UI adhering to Material 3 design principles.
17. Support media sharing including images, audio recordings, and files.
18. Implement user search, contact management, blocking, and privacy controls.
19. Provide offline message caching with SQLite on supported platforms.
20. Define and enforce Firestore security rules to protect user data.

### 1.4 Scope of the Project

In Scope:

- Android and web platform deployment.
- Real-time private and group chat.
- AI chatbot and sentiment-driven character animation.
- Stories (image, video, text) with 24-hour expiry.
- User profiles, avatars, search, and presence indicators.
- Media messages: image, audio, file.
- Message reactions, reply/quote, typing indicators.
- Privacy settings: mute, pin, block, read receipt visibility.
- Dark and light themes with glass-morphism effects.

Out of Scope:

- End-to-end encryption.
- Voice/video calling.
- iOS deployment (partially configured, not fully tested).
- Message translation (model prepared, UI not fully wired).
- Push notifications via FCM (architecture present, not deployed).

### 1.5 Report Structure

Chapter 2 reviews related literature. Chapter 3 defines requirements. Chapter 4 describes the system design. Chapter 5 covers database design. Chapter 6 details implementation. Chapter 7 addresses testing. Chapter 8 presents results and discussion. Chapter 9 concludes and proposes future work.

## 2. Literature Review

### 2.1 Evolution of Instant Messaging Applications

Instant messaging traces its origins to UNIX talk (1970s) and IRC (1988). The smartphone era brought WhatsApp (2009) and iMessage (2011), introducing persistent group threads and media sharing. Modern platforms (Telegram, Signal, Discord) added bots, channels, ephemeral messages, and end-to-end encryption. Contemporary research focuses on latency reduction, scalable architectures, and AI integration [1][2].

### 2.2 Cross-Platform Mobile Development Frameworks

Cross-platform development reduces cost and time-to-market by sharing a single codebase across platforms. The main approaches are:

- React Native (Meta, 2015): Bridges JavaScript to native components. Widely adopted but with bridge performance overhead.
- Flutter (Google, 2018): Uses the Dart language and renders directly via the Skia/Impeller GPU engine, bypassing native UI components entirely. Delivers near-native performance with pixel-perfect consistency across Android, iOS, web, and desktop [3].
- Xamarin/.NET MAUI (Microsoft): C#-based with native component wrapping.

Flutter was chosen for CHATZY due to its hot reload, rich widget ecosystem, Material 3 support, and proven Firebase integration.

### 2.3 Backend-as-a-Service (BaaS) Platforms

BaaS platforms abstract server infrastructure, allowing developers to focus on application logic. Key players include:

| Platform | Real-Time DB | Auth | Storage | Serverless |
|---|---|---|---|---|
| Firebase (Google) | Firestore / RTDB | Yes | Yes | Cloud Functions |
| Supabase | PostgreSQL | Yes | Yes | Edge Functions |
| AWS Amplify | DynamoDB | Yes | S3 | Lambda |
| Parse | MongoDB | Yes | Yes | Cloud Code |

Firebase Firestore was selected for CHATZY for its real-time listener model (snapshot streams), tight Flutter SDK integration (cloud_firestore package), generous free tier (Spark plan), and built-in offline persistence [4].

### 2.4 Artificial Intelligence in Chat Applications

AI features in messaging applications have grown rapidly:

- Smart Reply (Google, 2017): On-device ML to suggest short responses in Gmail and Messages.
- Bing Chat / Copilot (Microsoft, 2023): GPT-4 integration in Teams and Edge.
- Bard / Gemini (Google, 2023–2024): Gemini API enables third-party AI integration.

Sentiment analysis — classifying emotional tone in text — has applications in customer service, mental health monitoring, and interactive characters. Models range from rule-based lexicons (VADER) to transformer-based classifiers (BERT, RoBERTa). For this project, Gemini 2.0 Flash's instruction-following capability was used as a lightweight sentiment classifier [5].

### 2.5 Ephemeral Content in Social Messaging

Snapchat (2011) popularised the concept of self-destructing messages. Instagram Stories (2016) and WhatsApp Status (2017) extended this to 24-hour content windows. Research shows ephemeral content encourages more casual, authentic sharing behaviour compared to permanent posts [6]. CHATZY implements the 24-hour story paradigm with viewership tracking and reaction support.

### 2.6 Summary of Related Work

Existing open-source Flutter chat applications (e.g., FlutterFire chat samples, StreamChat Flutter) demonstrate Firebase integration but are either sample-scale or require paid SDK subscriptions. CHATZY extends this with: a complete glass-morphism UI system, AI sentiment analysis, animated 3D characters, and ephemeral stories — making it a more comprehensive educational implementation.

## 3. Requirements Analysis

### 3.1 Functional Requirements

Authentication (AUTH)

| ID | Requirement |
|---|---|

| | |
|---|---|
| AUTH-01 | The system shall allow users to register with email, password, display name, username, phone number, and an optional profile avatar. |
| AUTH-02 | The system shall validate that usernames are unique before account creation. |
| AUTH-03 | The system shall allow users to log in with either their email address or username. |
| AUTH-04 | The system shall allow users to reset their password via email. |
| AUTH-05 | The system shall maintain user sessions persistently across application restarts. |
| AUTH-06 | The system shall set user online status to true on login and false on logout/app close. |

Messaging (MSG)

| ID | Requirement |
|---|---|
| MSG-01 | The system shall deliver text messages in real time between two users. |
| MSG-02 | The system shall display message delivery status: sending → sent → delivered → read. |
| MSG-03 | The system shall display a typing indicator when the other participant is composing. |
| MSG-04 | The system shall support replying to (quoting) a previous message. |
| MSG-05 | The system shall allow users to react to messages with emoji. |
| MSG-06 | The system shall track and display unread message counts per conversation. |
| MSG-07 | The system shall support sending images, audio recordings, and files. |
| MSG-08 | The system shall support group chats with multiple participants and admin roles. |

AI Features (AI)

| ID | Requirement |
|---|---|
| AI-01 | The system shall provide an AI chatbot powered by Google Gemini 2.0 Flash. |
| AI-02 | The AI chatbot shall maintain conversation history context for coherent multi-turn dialogue. |
| AI-03 | The system shall perform sentiment analysis on messages to determine the user's mood. |
| AI-04 | The animated character's expression/animation shall update based on detected mood. |

Stories (STR)

| ID | Requirement |
|---|---|

| | |
|---|---|
| STR-01 | The system shall allow users to post stories containing images, videos, or text. |
| STR-02 | Stories shall automatically expire 24 hours after creation. |
| STR-03 | The system shall record which users have viewed each story. |
| STR-04 | Users shall be able to react to stories with emoji. |

Profile and Social (PRF)

| ID | Requirement |
|---|---|
| PRF-01 | Users shall be able to view and edit their profile (name, bio, avatar, phone). |
| PRF-02 | Users shall be able to search for other users by name or username. |
| PRF-03 | Users shall be able to block and unblock other users. |
| PRF-04 | Users shall be able to mute and pin individual chats. |
| PRF-05 | The system shall display real-time online/offline presence for other users. |

Settings (SET)

| ID | Requirement |
|---|---|
| SET-01 | Users shall be able to toggle between dark and light mode. |
| SET-02 | Users shall be able to choose a background style (Nebula gradient or Deep Black). |
| SET-03 | Users shall be able to configure AI feature preferences. |

## 3.2 Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| NFR-01 | Performance | The chat list and message stream shall load within 2 seconds on a stable internet connection. |
| NFR-02 | Scalability | The Firestore backend shall be capable of handling concurrent users via Google's auto-scaling infrastructure. |
| NFR-03 | Reliability | The application shall handle network interruptions gracefully, queuing messages as "pending" and retrying on reconnection. |
| NFR-04 | Security | All Firestore data access shall be governed by security rules requiring authentication. |
| NFR-05 | Usability | The UI shall conform to Material 3 guidelines and support both dark |

| | | |
|---|---|---|
| | | and light themes. |
| NFR-06 | Maintainability | The codebase shall follow a layered architecture separating models, services, providers, and screens. |
| NFR-07 | Portability | The application shall run on Android 6.0+ and modern web browsers without modification to the shared Dart codebase. |
| NFR-08 | Privacy | User passwords shall never be stored in the application; authentication tokens are managed exclusively by Firebase Auth. |

## 3.3 System Requirements

***Minimum Requirements to Run CHATZY (End User — Android)***

| Component | Minimum Requirement |
|---|---|
| Operating System | Android 6.0 (Marshmallow) — API Level 23 |
| RAM | 2 GB |
| Storage | 100 MB free space |
| Internet | Active internet connection (Wi-Fi or mobile data) |
| Camera / Microphone | Required for avatar upload, audio messages, and story creation |
| Screen Resolution | 720 × 1280 px or higher |

***Recommended Requirements (Android)***

| Component | Recommended |
|---|---|
| Operating System | Android 10.0 (API Level 29) or higher |
| RAM | 4 GB or more |
| Storage | 500 MB free space |
| Internet | Wi-Fi or 4G/5G |

***Development Environment Requirements***

| Tool | Minimum Version | Purpose |
|---|---|---|
| Flutter SDK | 3.0.0 | Build and run the application |
| Dart SDK | 3.0.0 | Language runtime |
| Android Studio | Hedgehog (2023.1) | IDE and Android SDK manager |
| Java JDK | 17 | Android build toolchain |
| Android SDK | API 21+ | Android compilation target |
| Git | 2.x | Version control |
| Internet Connection | Required | Firebase and Gemini API access |
| Firebase Project | Active project with Firestore, Auth, Storage enabled | Backend services |
| Google Gemini API Key | Valid key from Google AI Studio | AI features |

## 3.4 Use Case Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CHATZY SYSTEM                               │
│                                                                          │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────────────┐    │
│   │  Register    │     │   Login      │     │   Forgot Password    │    │
│   └──────────────┘     └──────────────┘     └──────────────────────┘    │
│                                                                          │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────────────┐    │
│   │ Send Msg     │     │ Send Media   │     │   React to Msg       │    │
│   └──────────────┘     └──────────────┘     └──────────────────────┘    │
│                                                                          │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────────────┐    │
│   │ Create       │     │ View Story   │     │   Post Story         │    │
│   │ Group Chat   │     │              │     │                      │    │
│   └──────────────┘     └──────────────┘     └──────────────────────┘    │
│                                                                          │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────────────┐    │
│   │  Chat with   │     │  Search      │     │   Block User         │    │
│   │  AI Bot      │     │  Users       │     │                      │    │
│   └──────────────┘     └──────────────┘     └──────────────────────┘    │
│                                                                          │
│   ┌──────────────┐     ┌──────────────┐                                 │
│   │  Edit        │     │  Change      │                                 │
│   │  Profile     │     │  Theme       │                                 │
│   └──────────────┘     └──────────────┘                                 │
└──────────────────────────────────────────────────────────────────────────┘
                                  ▲
                                  |
                          [Registered User]
```

## 3.5 Use Case Descriptions

UC-01: Send a Text Message

| Field | Detail |
|---|---|
| Use Case ID | UC-01 |
| Name | Send Text Message |
| Actor | Authenticated User |
| Precondition | User is logged in; a chat conversation exists. |
| Main Flow | 1. User opens a chat. 2. User types text in the input field. 3. User taps the send button. 4. Message appears immediately (optimistic update). 5. Message is persisted to Firestore. 6. Recipient receives the message in real time. |
| Postcondition | Message stored in Firestore with status "sent". Recipient's unread count incremented. |
| Alternate Flow | If network is unavailable, message is stored locally with "pending" status and retried when connection restores. |

UC-02: Chat with AI Bot

| Field | Detail |
|---|---|
| Use Case ID | UC-02 |
| Name | Chat with AI Chatbot |
| Actor | Authenticated User |
| Precondition | User is logged in; AI API key is configured. |
| Main Flow | 1. User navigates to the AI Chat from the chat list. 2. User types a message. 3. System sends message + conversation history to Gemini API. 4. AI response is displayed. 5. Sentiment of user message is analysed. 6. Animated character updates its expression. |
| Postcondition | AI response stored in local conversation history. |
| Alternate Flow | If API call fails, a generic fallback response is displayed. |

UC-03: Register a New Account

| Field | Detail |
|---|---|
| Use Case ID | UC-03 |
| Name | User Registration |
| Actor | New User (unauthenticated) |
| Precondition | User has no existing CHATZY account. |
| Main Flow | 1. User opens the app and taps "Register". 2. User enters display name, unique username, email, password, and optional phone number. 3. User optionally selects a profile avatar from the device gallery. 4. System checks username uniqueness in Firestore. 5. Firebase Auth creates the account. 6. Avatar is uploaded to Firebase Storage. 7. UserModel is saved to Firestore with searchKeywords generated. 8. User is redirected to HomeScreen. |
| Postcondition | New user document created in Firestore users collection; user is authenticated. |
| Alternate Flow | If username is already taken, an error message is shown and the user must choose a different username. |

UC-04: Post a Story

| Field | Detail |
|---|---|
| Use Case ID | UC-04 |
| Name | Post Ephemeral Story |
| Actor | Authenticated User |
| Precondition | User is logged in. |
| Main Flow | 1. User navigates to the Stories tab. 2. User taps the "+" button to create a new story. 3. User selects image, video, or text content. 4. User adds an optional caption. 5. System uploads media to Firebase Storage under stories/{userId}/. 6. Story |

| | |
|---|---|
| | document is created in Firestore with expiresAt = now + 24 hours. 7. Story appears in the Stories feed for all users. |
| Postcondition | Story document stored in Firestore and visible to all authenticated users for 24 hours. |
| Alternate Flow | If media upload fails, an error toast is shown and the story is not posted. |

UC-05: Create a Group Chat

| Field | Detail |
|---|---|
| Use Case ID | UC-05 |
| Name | Create Group Chat |
| Actor | Authenticated User |
| Precondition | User is logged in; at least two other users exist. |
| Main Flow | 1. User navigates to the Contacts screen. 2. User selects two or more contacts. 3. User taps "Create Group". 4. User enters a group name and optional avatar. 5. System creates a chat document in Firestore with type=group and adminIds containing the creator. 6. All participants receive the group chat in their Chats List in real time. |
| Postcondition | Group chat document stored in Firestore; all participants subscribed to the chat stream. |
| Alternate Flow | If fewer than two participants are selected, the system shows a validation error. |

UC-06: Block a User

| Field | Detail |
|---|---|
| Use Case ID | UC-06 |
| Name | Block User |
| Actor | Authenticated User |
| Precondition | User is logged in; target user exists. |
| Main Flow | 1. User opens the target user's profile via the Contacts or Chat screen. 2. User taps "Block User". 3. System updates the isBlocked field on the target user's Firestore document. 4. Target user is added to the current user's block list. 5. Confirmation message is shown. |
| Postcondition | Blocked user no longer appears in the user's chat and contact views. |
| Alternate Flow | User may unblock at any time via the Block List screen in Privacy Settings. |

# 4. System Design and Architecture

## 4.1 High-Level Architecture

CHATZY follows a client-cloud architecture. The Flutter application runs on the client device and communicates exclusively with Google Firebase services and the Gemini API. There is no custom application server.

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CLIENT (Flutter App)                          │
│  ┌──────────────┐   ┌──────────────────┐   ┌──────────────────────────┐ │
│  │   Screens    │   │   Providers      │   │        Services          │ │
│  │  (Flutter    │→  │  (State Mgmt)    │→  │  Auth / Firestore /      │ │
│  │  Widgets)    │   │  ChangeNotif.    │   │  Storage / AI / SQLite   │ │
│  └──────────────┘   └──────────────────┘   └──────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │  HTTPS / WebSocket
                                │
          ┌─────────────────────┼──────────────────────────────┐
          ▼                     ▼                              ▼

┌────────────────┐   ┌────────────────────┐   ┌──────────────────────────┐
│  Firebase      │   │  Cloud Firestore   │   │  Firebase Storage        │
│  Auth          │   │  (NoSQL DB,        │   │  (Binary: Images,        │
│                │   │   Real-Time        │   │   Audio, Avatars,        │
│                │   │   Streams)         │   │   Stories)               │
└────────────────┘   └────────────────────┘   └──────────────────────────┘
                                                          │
                                                          ▼
                                                ┌────────────────────┐
                                                │  Google Gemini API │
                                                │  (AI Chatbot &     │
                                                │   Sentiment)       │
                                                └────────────────────┘
```

## 4.2 Technology Stack

| Layer | Technology | Version |
|---|---|---|
| Language | Dart | 3.5.4 |
| UI Framework | Flutter | 3.x |
| State Management | Provider (ChangeNotifier) | 6.1.1 |
| Authentication | Firebase Auth | 6.2.0 |
| Real-Time Database | Cloud Firestore | 6.1.3 |
| File Storage | Firebase Storage | 13.1.0 |
| App Security | Firebase App Check | 0.4.1+5 |
| AI / LLM | Google Gemini 2.0 Flash | google_generative_ai 0.4.7 |
| Local Cache | SQLite (sqflite) | 2.4.1 |
| Local Prefs | SharedPreferences | 2.3.5 |
| UI Fonts | Google Fonts | 6.1.0 |
| Animation | flutter_animate, Lottie | 4.3.0, 3.1.0 |
| 3D Rendering | flutter_cube, o3d | 0.1.1, 3.1.3 |
| Media Playback | media_kit | 1.1.10+1 |
| Image Picking | image_picker | 1.1.2 |
| Audio Recording | record | 6.2.0 |
| Build Target | Android, Web | Android API 21+ |

## 4.3 Application Layer Architecture

The application is organized into five distinct layers:

```
┌──────────────────────────────────────────────────────┐
│                PRESENTATION LAYER                    │
│  Screens, Widgets, Navigation, Theming               │
├──────────────────────────────────────────────────────┤
│              STATE MANAGEMENT LAYER                  │
│  ChatProvider, AuthService, StoryProvider,           │
│  ThemeProvider, CharacterProvider                    │
├──────────────────────────────────────────────────────┤
│                 SERVICE LAYER                        │
│  FirestoreService, AuthService, StorageService,      │
│  AiService, StoryService, DatabaseService            │
├──────────────────────────────────────────────────────┤
│                  MODEL LAYER                         │
│  UserModel, ChatModel, MessageModel, StoryModel      │
├──────────────────────────────────────────────────────┤
│              INFRASTRUCTURE LAYER                    │
│  Firebase SDK, Gemini SDK, SQLite, SharedPrefs       │
└──────────────────────────────────────────────────────┘
```

Layer Responsibilities:

- Presentation: Flutter widgets that render UI and respond to user input. Screens use Consumer and Provider.of to read from state management.
- State Management: ChangeNotifier subclasses that hold application state, orchestrate service calls, and notify the UI of changes.
- Service: Encapsulated wrappers around external SDKs. Each service has a single responsibility (auth, Firestore, storage, AI).
- Model: Plain Dart objects representing domain entities. Each model implements toMap() / fromMap() for Firestore serialization.
- Infrastructure: Third-party SDKs and platform APIs, accessed only through the service layer.

## 4.4 Navigation Architecture

CHATZY uses Flutter's Navigator 1.0 with named routes for the top-level flow, combined with push/pop for contextual navigation.

```
MaterialApp (root)
│
├─ / → AuthWrapper
│       ├─ (unauthenticated) → /login → LoginScreen
│       │                    → /register → RegisterScreen
│       └─ (authenticated) → /home → HomeScreen
│
HomeScreen (BottomNavigationBar — 4 tabs)
├─ Tab 0: ChatsListScreen
│   ├─ push → ChatScreen
│   ├─ push → GroupChatScreen
│   └─ push → AiChatbotScreen
├─ Tab 1: ContactsScreen
│   └─ push → UserDetailsScreen
```

```
├─ Tab 2: StoriesFeedScreen
│    ├─ push → StoryViewerScreen
│    └─ push → CreateStoryScreen
└─ Tab 3: SettingsScreen
     ├─ push → ProfileScreen → EditProfileScreen
     ├─ push → ThemeSettingsScreen
     ├─ push → NotificationsScreen
     ├─ push → PrivacySettingsScreen
     ├─ push → AiAccountSettingsScreen
     ├─ push → AboutScreen
     └─ push → BlockListScreen
```

## 4.5 State Management Design

The Provider pattern (based on ChangeNotifier) was selected for its simplicity, testability, and official Flutter recommendation for medium-complexity applications.

Providers registered at app root (MultiProvider):

| Provider | Scope | Responsibility |
|---|---|---|
| AuthService | Global | Firebase Auth state, current user model, online status |
| ChatProvider | Global | Chats list, messages per chat, presence, Firestore subscriptions |
| StoryProvider | Global | Active stories, expiry filtering, story actions |
| ThemeProvider | Global | Dark/light mode, background style, theme data |
| CharacterProvider | Global | AI character style, mood, animation state |

Data Flow:

```
User Action (Widget)
       │
       ▼
Provider Method Called
       │
       ├─→ Service Layer (Firestore / Auth / AI)
       │          │
       │          └─→ Firebase / Gemini API
       │                    │
       │          ←──────────
       ├─ State Updated
       └─ notifyListeners()
                  │
                  ▼
         Widgets Rebuild
```

## 4.6 Firebase Architecture

CHATZY uses three Firebase services:

Firebase Authentication: Manages identity using email/password. Provides a persistent session token (User object) accessible via FirebaseAuth.instance.authStateChanges() stream. Password reset is handled by sending a reset email through Firebase's built-in mechanism.

Cloud Firestore: A NoSQL document-oriented database organized into collections and subcollections. Firestore's real-time listener (snapshots()) is used throughout CHATZY to push changes to the UI without polling. The data model is described in Chapter 5.

Firebase Cloud Storage: Stores binary assets (profile images, chat media, story media) in a hierarchical path structure. Files are referenced by their download URL, which is stored in Firestore documents.

Firebase App Check: Enabled for Android (PlayIntegrity) to verify that requests originate from the genuine CHATZY application, protecting the backend from abuse.

### 4.7 AI Module Architecture

```
User Message (text)
       │
       ├─→ AiService.getChatResponse()
       │        │
       │        ├─ Build prompt with conversation history context
       │        └─ Call Gemini 2.0 Flash → Return response text
       │
       └─→ AiService.analyzeSentiment()
                │
                ├─ Classify message into: happy / sad / thinking /
                │  waving / surprised / angry / sleeping / neutral
                └─ Update CharacterProvider.mood
                         │
                         └─ AnimatedCharacterWidget rebuilds
```

The AI module is designed to be fault-tolerant: if the Gemini API returns an error, getChatResponse() returns a generic fallback string, and analyzeSentiment() falls back to keyword matching.

## 5. Database Design

### 5.1 Cloud Firestore Data Model

Firestore uses a collections-documents-subcollections hierarchy. CHATZY defines four top-level collections.

***Collection: users***

Stores user profile data. Each document ID is the Firebase Auth UID.

| Field | Type | Description |
|---|---|---|
| id | String | Firebase UID (primary key) |
| name | String | Display name |
| email | String | Registered email |
| username | String | Unique username for login/search |
| avatar | String? | Profile image download URL |
| bio | String? | User biography |
| phone | String? | Phone number |

| | | |
|---|---|---|
| isOnline | Integer (0/1) | Current online status |
| lastSeen | String | ISO 8601 timestamp of last activity |
| isBlocked | Integer (0/1) | Global block flag |
| customNickname | String? | Nickname visible only to the setting user |
| isMuted | Integer (0/1) | Mute flag |
| searchKeywords | Array\<String\> | Lowercase prefixes for search optimization |

Search Keyword Generation: For a user named "Alice Smith" with username "alice99", the keywords array is generated as all substrings: ["a", "al", "ali", "alic", "alice", "a", "al", "ali", "alic", "alice", "smith", "alice99", ...]. This enables Firestore array-contains queries for prefix search without a dedicated search service.

***Collection: chats***

Stores chat metadata. Each document represents one conversation (private or group).

| Field | Type | Description |
|---|---|---|
| id | String | Auto-generated document ID |
| name | String | Chat display name |
| type | Integer | 0=private, 1=group, 2=AI |
| participants | Array\<Object\> | Snapshot of participant UserModels |
| participantIds | Array\<String\> | Array of UIDs for query filtering |
| unreadCount | Map\<String, Integer\> | Per-user unread message count |
| isPinned | Boolean | Whether chat is pinned to top |
| isMuted | Boolean | Whether notifications are muted |
| lastMessage | Object | Snapshot of the most recent message |
| typingUserId | String? | UID of user currently typing |
| adminIds | Array\<String\> | UIDs with admin permissions (groups) |
| createdAt | Timestamp | Creation time |
| updatedAt | Timestamp | Last modification time |

***Subcollection: chats/{chatId}/messages***

Stores all messages for a given chat.

| Field | Type | Description |
|---|---|---|
| id | String | Auto-generated message ID |
| chatId | String | Parent chat ID |
| senderId | String | UID of sender |
| content | String | Message text or media download URL |
| type | Integer | 0=text, 1=image, 2=video, 3=audio, 4=file, 5=sticker, 6=aiSuggestion, 7=translated |

| | | |
|---|---|---|
| status | Integer | 0=sending, 1=sent, 2=delivered, 3=read, 4=failed, 5=pending |
| timestamp | Timestamp | Send time |
| replyToId | String? | ID of message being quoted |
| translatedContent | String? | Translated text if applicable |
| reactions | Map\<String, String\> | userId → emoji mapping |
| aiSuggestion | String? | AI-generated suggestion text |

***Collection: stories***

Stores ephemeral content items.

| Field | Type | Description |
|---|---|---|
| id | String | Auto-generated document ID |
| userId | String | Creator UID |
| userName | String | Creator display name at post time |
| userAvatar | String? | Creator avatar URL at post time |
| type | Integer | 0=image, 1=video, 2=text |
| content | String | Media URL or text content |
| caption | String? | Optional caption |
| createdAt | Integer | Unix milliseconds |
| expiresAt | Integer | Unix milliseconds (createdAt + 86,400,000) |
| viewedBy | Array\<String\> | UIDs who viewed the story |
| reactions | Array\<String\> | Emoji reactions |
| backgroundColor | String? | Hex color for text stories |
| textColor | String? | Hex color for text overlay |

## 5.2 Firebase Storage Structure

```
gs://chitzy-7ce77.firebasestorage.app/
│
├── users/
│   └── {userId}/
│       └── avatars/
│           └── {timestamp}_{filename}
│
├── chats/
│   └── {chatId}/
│       ├── images/
│       │   └── {timestamp}_{filename}
│       └── audio/
│           └── {timestamp}_{filename}
│
└── stories/
    └── {userId}/
        └── {timestamp}_{filename}
```

All uploaded files are named with a Unix timestamp prefix to guarantee uniqueness. The resulting download URL is stored as the content or avatar field in the corresponding Firestore document.

## 5.3 Local SQLite Cache

On Windows (and Linux), CHATZY uses an SQLite database via the sqflite_common_ffi package to cache chat and message data for offline access.

Table: chats

```
CREATE TABLE chats (
  id        TEXT PRIMARY KEY,
  data      TEXT NOT NULL  -- JSON-encoded ChatModel
);
```

Table: messages

```
CREATE TABLE messages (
  id        TEXT PRIMARY KEY,
  chatId    TEXT NOT NULL,
  data      TEXT NOT NULL  -- JSON-encoded MessageModel
);
```

The local cache is treated as a secondary source. When Firestore streams deliver data, it is simultaneously written to SQLite. When the app is offline, data is read from SQLite and displayed while the connection is re-established.

## 5.4 Security Rules

CHATZY enforces access control at the Firestore level using declarative security rules. The rules are defined in firestore.rules and deployed to the Firebase project.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Users collection: anyone can read profiles (for search),
    // but only the authenticated owner can write their own document.
    match /users/{userId} {
      allow read: if true;
      allow write: if request.auth != null
                   && request.auth.uid == userId;
    }

    // Chats and messages: any authenticated user can read/write.
    // In production this should be restricted to participants only.
    match /chats/{chatId} {
      allow read, write: if request.auth != null;
    }
    match /messages/{messageId} {
      allow read, write: if request.auth != null;
    }

    // Stories: authenticated users can read and create.
    // Only the story's creator can update or delete.
    match /stories/{storyId} {
      allow read:   if request.auth != null;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
                            && request.auth.uid == resource.data.userId;
```

```
    }

    // Contacts: authenticated read/write.
    match /contacts/{contactId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

# 6. Implementation

## 6.1 Development Environment

| Tool | Version | Purpose |
|---|---|---|
| Flutter SDK | 3.x | Framework and build toolchain |
| Dart SDK | 3.5.4 | Language runtime |
| Android Studio / VS Code | Latest | IDE with Flutter extension |
| Firebase Console | Web | Firebase project configuration |
| Google AI Studio | Web | Gemini API key management |
| Git | 2.x | Version control |
| Android Emulator / Physical Device | API 21+ | Testing |

## 6.2 Project Structure

```
chatzy/
├── lib/
│   ├── main.dart                   # Entry point, DI, routing
│   ├── firebase_options.dart       # Auto-generated Firebase config
│   ├── models/
│   │   ├── user_model.dart
│   │   ├── chat_model.dart
│   │   ├── message_model.dart
│   │   └── story_model.dart
│   ├── services/
│   │   ├── auth_service.dart
│   │   ├── user_service.dart
│   │   ├── firestore_service.dart
│   │   ├── firebase_storage_service.dart
│   │   ├── ai_service.dart
│   │   ├── story_service.dart
│   │   ├── database_service.dart
│   │   ├── storage_service.dart
│   │   └── mock_auth_service.dart
│   ├── providers/
│   │   ├── chat_provider.dart
│   │   ├── story_provider.dart
│   │   ├── theme_provider.dart
│   │   └── character_provider.dart
│   ├── screens/
│   │   ├── auth/            # SplashScreen, LoginScreen, RegisterScreen
│   │   ├── home/            # HomeScreen, ChatsListScreen, ContactsScreen,
│   │   │                    # StoriesFeedScreen, SettingsScreen
```

```
│   │   ├── chat/           # ChatScreen, GroupChatScreen, AiChatbotScreen,
│   │   │                   # ChatSearchScreen, ChatSettingsScreen,
MediaGallery
│   │   ├── profile/        # ProfileScreen, EditProfileScreen,
│   │   │                   # UserDetailsScreen, BlockListScreen
│   │   ├── settings/       # ThemeSettings, Notifications, Privacy,
│   │   │                   # AiAccountSettings, About
│   │   └── stories/        # CreateStoryScreen, StoryViewerScreen
│   ├── widgets/
│   │   ├── glass_container.dart
│   │   ├── glass_button.dart
│   │   ├── glass_text_field.dart
│   │   ├── animated_human_character.dart
│   │   ├── character_3d.dart
│   │   ├── huggy_3d_character.dart
│   │   ├── audio_message_bubble.dart
│   │   ├── voice_recorder.dart
│   │   ├── swipe_to_reply.dart
│   │   ├── poll_widget.dart
│   │   └── liquid_glass/   # Shader-based glass effects
│   └── theme/
│       └── app_theme.dart  # Colors, typography, Material 3 theme
├── assets/
│   ├── images/             # Backgrounds, icons
│   ├── animations/         # Lottie JSON animations
│   └── models/             # 3D GLB files (HuggyWuggy.glb, etc.)
├── android/                # Android platform configuration
├── web/                    # Web platform configuration
├── firestore.rules         # Firestore security rules
└── pubspec.yaml            # Dependencies and assets manifest
```

## 6.3 Authentication Module

Registration Flow:

```
// lib/services/auth_service.dart (simplified)
Future<UserModel?> signUp({
  required String email,
  required String password,
  required String name,
  required String username,
  String? phone,
  File? avatarFile,
}) async {
  // 1. Check username uniqueness
  bool taken = await _userService.isUsernameTaken(username);
  if (taken) throw Exception('Username already taken');

  // 2. Create Firebase Auth account
  UserCredential cred = await _auth.createUserWithEmailAndPassword(
    email: email, password: password);

  // 3. Upload avatar if provided
  String? avatarUrl;
  if (avatarFile != null) {
    avatarUrl = await _storageService.uploadUserAvatar(
```

```
      cred.user!.uid, avatarFile);
  }

  // 4. Build and save UserModel to Firestore
  UserModel user = UserModel(
    id: cred.user!.uid,
    name: name, email: email, username: username,
    avatar: avatarUrl, phone: phone, isOnline: true,
    searchKeywords: _generateKeywords(name, username),
  );
  await _userService.saveUser(user);
  return user;
}
```

Session Management: AuthWrapper listens to AuthService.userStream (a Stream<UserModel?>) that merges Firebase Auth state changes with real-time Firestore user profile updates. If userStream emits null, the app navigates to LoginScreen; if it emits a UserModel, it navigates to HomeScreen.

## 6.4 Real-Time Messaging Module

Message Sending with Optimistic Update:

```
// lib/providers/chat_provider.dart (simplified)
Future<void> addMessage(String chatId, MessageModel message,
    {File? mediaFile}) async {
  // 1. Optimistic update — show message immediately
  message = message.copyWith(status: MessageStatus.sending);
  _messages[chatId]?.insert(0, message);
  notifyListeners();

  // 2. Upload media if present
  String content = message.content;
  if (mediaFile != null) {
    content = await _storageService.uploadChatMedia(chatId, mediaFile);
  }

  // 3. Persist to Firestore
  final saved = await _firestoreService.sendMessage(
    message.copyWith(content: content, status: MessageStatus.sent));

  // 4. Replace optimistic entry with persisted version
  final idx = _messages[chatId]
      ?.indexWhere((m) => m.id == message.id) ?? -1;
  if (idx != -1) {
    _messages[chatId]![idx] = saved;
    notifyListeners();
  }
}
```

Real-Time Message Stream:

```
// Subscribe to live message updates for a chat
void subscribeToMessages(String chatId) {
  _messageSubscriptions[chatId] = _firestoreService
      .getChatMessages(chatId)
      .listen((messages) {
```

```
        _messages[chatId] = messages;
        notifyListeners();
      });
}
```

Message Status Lifecycle: Each message progresses through statuses managed in Firestore:

```
[User sends] → sending (optimistic, local only)
           → sent     (written to Firestore successfully)
           → delivered (recipient's app received the message)
           → read      (recipient opened the chat)
```

## 6.5 AI Chatbot Module

Sentiment Analysis:

```
// lib/services/ai_service.dart (simplified)
Future<String> analyzeSentiment(String message) async {
  final model = GenerativeModel(
    model: 'gemini-2.0-flash', apiKey: _apiKey);
  final response = await model.generateContent([
    Content.text(
      'Analyze the sentiment of this message and respond with '
      'exactly one word from: happy, sad, thinking, waving, '
      'surprised, angry, sleeping, neutral.\n\nMessage: $message')
  ]);
  return response.text?.trim().toLowerCase() ?? 'neutral';
}
```

AI Chat with History:

```
Future<String> getChatResponse(String userMessage,
    List<Map<String, String>> history) async {
  final model = GenerativeModel(
    model: 'gemini-2.0-flash',
    apiKey: _apiKey,
    systemInstruction: Content.text(
      'You are CHATZY AI, a helpful personal assistant.'));

  final contents = [
    ...history.map((h) => Content(h['role']!, [TextPart(h['text']!)])),
    Content.text(userMessage),
  ];

  final response = await model.generateContent(contents);
  return response.text ?? 'I could not process that. Please try again.';
}
```

## 6.6 Stories Module

Stories are implemented using a combination of Firestore and Firebase Storage:

21. Create Story: User selects media → uploaded to stories/{userId}/{timestamp} in Firebase Storage → download URL stored in Firestore stories collection with expiresAt = createdAt + 86_400_000ms.
22. Display Stories: StoryService.getActiveStories() queries Firestore for documents where expiresAt > now using a Firestore stream. Results are grouped by userId in StoryProvider.latestPerUser.

23. View Tracking: When a user views a story, StoryService.viewStory() uses Firestore arrayUnion to add the viewer's UID to viewedBy.
24. Expiry Cleanup: Client-side filtering ensures expired stories (where expiresAt < DateTime.now().millisecondsSinceEpoch) are never displayed, even if not yet deleted from Firestore. A Cloud Function can be added in future to clean up expired documents server-side.

### 6.7 User Interface and Theming

Glass-Morphism Container:

The GlassContainer widget is a reusable component that wraps Flutter's BackdropFilter to create frosted glass effects:

```
// lib/widgets/glass_container.dart (simplified)
class GlassContainer extends StatelessWidget {
  final Widget child;
  final double blur;
  final double opacity;
  final Color? color;
  final BorderRadius? borderRadius;

  @override
  Widget build(BuildContext context) {
    return ClipRRect(
      borderRadius: borderRadius ?? BorderRadius.circular(20),
      child: BackdropFilter(
        filter: ImageFilter.blur(sigmaX: blur, sigmaY: blur),
        child: Container(
          decoration: BoxDecoration(
            color: (color ?? Colors.white).withOpacity(opacity),
            borderRadius: borderRadius,
            border: Border.all(
              color: Colors.white.withOpacity(0.2), width: 1),
          ),
          child: child,
        ),
      ),
    );
  }
}
```

Theme System: AppTheme defines a centralized color palette, typography scale, and Material 3 ThemeData. ThemeProvider toggles between AppTheme.darkTheme and AppTheme.lightTheme, and selects the background decoration (Nebula gradient or Deep Black solid).

Color Palette:

| Role | Color | Hex |
|---|---|---|
| Primary Background | Pure Black | #000000 |
| Accent (iOS Blue) | Vibrant Blue | #2997FF |
| Accent Alt | Vibrant Red | #FF375F |
| Text Primary | White | #FFFFFF |
| Text Secondary | Light Gray | #EBEBF5 |
| Success | Green | #30D158 |

| | | |
|---|---|---|
| Error | Red | #FF453A |
| Warning | Orange | #FF9F0A |

### 6.8 Media Handling

Media messages follow this flow:

```
User picks file (image_picker / file_picker)
        |
        ▼
File validated (type, size check)
        |
        ▼
FirebaseStorageService.uploadChatImage/Audio/File()
  → Firebase Storage path: chats/{chatId}/{type}/{timestamp}_{name}
  → Returns download URL
        |
        ▼
MessageModel created with:
  - content = download URL
  - type = MessageType.image / audio / file
        |
        ▼
Saved to Firestore → Displayed in chat with appropriate bubble
```

Audio messages use the record package for in-app recording and audioplayers for playback within AudioMessageBubble.

### 6.9 Offline Support

When Firestore is unavailable:

25. Firestore SDK built-in caching: Firestore's native offline persistence (PersistenceSettings) caches documents and queues writes automatically on Android and iOS.
26. SQLite cache (Windows): DatabaseService saves chats and messages to a local SQLite database. On startup, ChatProvider loads cached data before Firestore streams respond.
27. Pending messages: Messages sent while offline are stored with MessageStatus.pending and automatically retried when connectivity is restored.

## 7. Testing and Evaluation

### 7.1 Testing Strategy

Given the scope of a final year project, the testing strategy focuses on manual functional testing and integration verification against the live Firebase backend. Automated unit tests are recommended as future work.

| Testing Type | Approach |
|---|---|
| Functional Testing | Manual test cases executed against Android device/emulator |
| Integration Testing | Verified Firebase Auth, Firestore reads/writes, Storage uploads |

| | |
|---|---|
| UI Testing | Manual walkthrough of all screens |
| AI Testing | Verified Gemini API responses and sentiment classification |
| Security Testing | Verified Firestore rules reject unauthorized access |

## 7.2 Functional Test Cases

| TC ID | Module | Test Description | Steps | Expected Result | Pass/Fail |
|---|---|---|---|---|---|
| TC-01 | Auth | Register new user | Enter valid email, password, unique username → tap Register | Account created, redirected to Home | Pass |
| TC-02 | Auth | Register with duplicate username | Enter existing username → tap Register | Error: "Username already taken" shown | Pass |
| TC-03 | Auth | Login with email | Enter valid email + password → tap Login | Authenticated, redirected to Home | Pass |
| TC-04 | Auth | Login with username | Enter username + password → tap Login | Email resolved, authenticated | Pass |
| TC-05 | Auth | Incorrect password | Enter wrong password → tap Login | Error: "Wrong password" shown | Pass |
| TC-06 | Messaging | Send text message | Open chat → type message → tap Send | Message appears in both sender and receiver chat | Pass |
| TC-07 | Messaging | Message delivery status | Send message → check status icons | Status progresses: sending → sent → delivered → read | Pass |
| TC-08 | Messaging | Typing indicator | User A types in chat to User B | User B sees "typing..." indicator | Pass |
| TC-09 | Messaging | Send image | Tap media icon → pick image → send | Image displayed as bubble with thumbnail | Pass |
| TC-10 | Messaging | React to message | Long-press message → select emoji | Reaction appears below message for both users | Pass |
| TC-11 | Messaging | Group chat | Create group with 3 users → | All 3 users receive the | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | | | send message | message | |
| TC-12 | AI | AI chatbot response | Open AI chat → type question | Coherent AI response displayed | Pass |
| TC-13 | AI | Sentiment → character mood | Type "I am so happy!" in AI chat | Character shows happy animation | Pass |
| TC-14 | Stories | Post image story | Create story → pick image → post | Story appears in Stories feed | Pass |
| TC-15 | Stories | Story expiry | Set expiresAt to 1 minute → wait | Story disappears after expiry | Pass |
| TC-16 | Stories | View tracking | User B views User A's story | User A sees User B in viewedBy list | Pass |
| TC-17 | Profile | Edit profile | Change display name → save | Updated name reflected in chat list and profile | Pass |
| TC-18 | Profile | Search user | Type partial name in search | Matching users shown in list | Pass |
| TC-19 | Privacy | Block user | Block User B | User B cannot see User A's messages | Pass |
| TC-20 | UI | Theme toggle | Switch from Dark to Light mode | All screens render in light palette | Pass |

## 7.3 Performance Considerations

- Message Pagination: getChatMessages() uses limit(100) on the Firestore query to prevent loading the entire message history on open. Pagination (loading older messages on scroll-up) is a planned improvement.
- Image Caching: CachedNetworkImage is used throughout to cache remote images locally, preventing redundant network requests and reducing load time for avatars and media thumbnails.
- Presence Subscriptions: ChatProvider manages _presenceSubscriptions to subscribe only to participants in open chats. Subscriptions are cleaned up on chat close to prevent memory leaks from excessive listeners.
- Search Keywords: Instead of a full-text search service, CHATZY uses Firestore array-contains on pre-generated keyword arrays, avoiding expensive full collection scans.

## 7.4 Security Evaluation

| Concern | Mitigation |
|---|---|
| Unauthenticated data access | Firestore rules require request.auth != null for all sensitive collections |
| User modifying another user's profile | Firestore rule: request.auth.uid == userId for user |

| | |
|---|---|
| | writes |
| Story modification by non-owner | Firestore rule: request.auth.uid == resource.data.userId for update/delete |
| Password storage | Passwords are never stored in the app or Firestore; Firebase Auth manages hashed credentials |
| API key exposure | Gemini API key is in source code (dev convenience); should be moved to environment variables or a proxy server before production deployment |
| App Check | Firebase App Check (PlayIntegrity) verifies requests come from the genuine app binary |

# 8. Results and Discussion

## 8.1 Feature Completion Summary

| Feature | Status | Notes |
|---|---|---|
| User Registration & Login | Complete | Email and username login both work |
| Real-Time Private Messaging | Complete | Full delivery/read receipt lifecycle |
| Group Messaging | Complete | Admin controls, participant management |
| AI Chatbot (Gemini) | Complete | Multi-turn conversation with history |
| Sentiment-Driven Character | Complete | 8-mood animation states |
| Stories (24h expiry) | Complete | Image, video, text stories with view tracking |
| Media Messages (image, audio, file) | Complete | Firebase Storage upload and display |
| Voice Message Recording | Complete | In-app recorder widget |
| Message Reactions | Complete | Per-user emoji mapping in Firestore |
| Reply/Quote Messages | Complete | Linked message preview in bubble |
| Typing Indicators | Complete | Firestore-backed real-time indicator |
| User Search | Complete | Prefix keyword search |
| Online Presence | Complete | Real-time isOnline + lastSeen |
| Block/Unblock Users | Complete | Privacy settings screen |
| Mute/Pin Chats | Complete | Per-chat settings |
| Dark/Light Theme | Complete | Persistent across sessions |
| Glass-Morphism UI | Complete | BackdropFilter throughout |
| 3D Animated Character | Complete | GLB model with flutter_cube |
| SQLite Offline Cache | Complete | Windows/Linux platforms |
| Message Translation | Partial | Model ready, UI not wired |
| Push Notifications (FCM) | Partial | Architecture present, not deployed |
| End-to-End Encryption | Not implemented | Planned for future version |

## 8.2 Challenges and Solutions

Challenge 1: Real-Time Presence at Scale

Firestore charges per document read. Subscribing to every user's presence document would generate excessive reads.

Solution: ChatProvider subscribes only to participants in currently open chats via getLiveUser(). Subscriptions are cancelled when the chat is closed.

Challenge 2: Message Ordering

Firestore documents are returned in insertion order by default, but network delays can cause out-of-order delivery.

Solution: All messages are queried with .orderBy('timestamp', descending: true) and displayed in reverse to show newest first, ensuring consistent ordering regardless of insertion time.

Challenge 3: Username Uniqueness

Firebase Auth only guarantees email uniqueness. Usernames are stored in Firestore.

Solution: Before account creation, UserService.isUsernameTaken() queries Firestore for any user with the same username. This is a best-effort check (not transactionally atomic), which is acceptable for the project scope.

Challenge 4: Gemini API Latency

AI responses add latency to the chatbot interaction.

Solution: A loading spinner is displayed while the API call is in-flight, and a timeout with fallback response ensures the UI never hangs indefinitely.

Challenge 5: Cross-Platform Media Handling

image_picker and file_picker behave differently on Android vs. Web (File objects vs. Bytes).

Solution: FirebaseStorageService provides two upload methods: uploadChatImage() (accepts File) and uploadChatImageBytes() (accepts Uint8List) to handle both platforms.

## 8.3 Limitations

28. No End-to-End Encryption: Messages are stored in plaintext in Firestore. Firebase's server-side encryption at rest is applied but messages are readable by Google. True E2E encryption (as in Signal) requires a key exchange protocol beyond this project's scope.
29. Firestore Security Rules Too Permissive: The current chat/message rules allow any authenticated user to read any chat. In production, rules should verify request.auth.uid in resource.data.participantIds.
30. No Pagination for Messages: The 100-message limit means very long conversations are truncated. Infinite scroll pagination is a required improvement.
31. Gemini API Key in Source: The API key is hardcoded for development convenience. Production deployment requires server-side key management or a dedicated proxy.
32. No Push Notifications: FCM integration is not deployed, meaning users must have the app open to receive messages.

# 9. Conclusion and Future Work

## 9.1 Conclusion

This project successfully designed, implemented, and tested CHATZY — a full-featured, cross-platform instant messaging application built with Flutter and Firebase. The application demonstrates practical mastery of mobile UI development, cloud-based real-time data synchronization, state management patterns, AI API integration, and database design.

All primary objectives were achieved:

- Secure user authentication with email and username support was implemented.
- Real-time messaging with full delivery status tracking was built using Firestore streams.
- An AI chatbot powered by Google Gemini 2.0 Flash was integrated with conversation history.
- Sentiment analysis drives real-time animated character expressions.
- A 24-hour ephemeral stories feature was implemented.
- A glass-morphism UI with dark/light theming provides a professional user experience.
- Firestore security rules protect user data and story ownership.

The resulting application is functionally comparable to commercial messaging applications and represents a significant demonstration of full-stack mobile development skills.

## 9.2 Future Work

The following enhancements are recommended for future development:

33. End-to-End Encryption: Implement the Signal Protocol or similar for message confidentiality.
34. Refined Firestore Security Rules: Restrict chat access to verified participants only.
35. Push Notifications (FCM): Deploy Firebase Cloud Messaging for background message delivery.
36. Message Pagination: Implement cursor-based pagination for message history scrolling.
37. Voice and Video Calling: WebRTC integration (e.g., Agora SDK) for real-time calls.
38. Message Translation: Wire the existing translation model to the UI with language auto-detection.
39. Scheduled Story Cleanup: Deploy a Firebase Cloud Function to delete expired stories from Firestore automatically.
40. Automated Testing: Implement unit tests for services and providers, and widget tests for critical UI paths.
41. iOS Deployment: Complete the iOS configuration and submit to the App Store.
42. Server-Side Gemini Proxy: Move API key management to a Cloud Function to prevent key exposure.

# 10. References

[1] Westlund, O. and Quinn, K. (2018). Mobile communication: A selected bibliography. Annals of the International Communication Association, 42(2), pp. 116–133.

[2] Lunden, I. (2020). "WhatsApp hits 2 billion users." TechCrunch. Available at: https://techcrunch.com/2020/02/12/whatsapp-hits-2-billion-users/

[3] Google LLC (2023). Flutter Documentation: Technical Overview. Available at: https://docs.flutter.dev/resources/architectural-overview

[4] Firebase Documentation (2024). Cloud Firestore Data Model. Google LLC. Available at: https://firebase.google.com/docs/firestore/data-model

[5] Google DeepMind (2024). Gemini API Documentation: Quickstart. Available at: https://ai.google.dev/gemini-api/docs/get-started

[6] Bayer, J.B., Ellison, N.B., Schoenebeck, S.Y. and Falk, E.B. (2016). "Sharing the small moments: ephemeral social interaction on Snapchat." Information, Communication & Society, 19(7), pp. 956–977.

[7] Martin, R.C. (2008). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

[8] Flutter Team (2024). Provider Package Documentation. Available at: https://pub.dev/packages/provider

[9] Google LLC (2024). Firebase Security Rules Reference. Available at: https://firebase.google.com/docs/rules

[10] Android Developers (2024). Android Manifest Permissions. Available at: https://developer.android.com/reference/android/Manifest.permission

## 11. Appendices

### Appendix A — Full Dependency List

```
# pubspec.yaml — Dependencies

dependencies:
  flutter:
    sdk: flutter

  # UI & Animation
  cupertino_icons: ^1.0.8
  google_fonts: ^6.1.0
  flutter_animate: ^4.3.0
  cached_network_image: ^3.3.1
  shimmer: ^3.0.0
  flutter_staggered_animations: ^1.1.1
  lottie: ^3.1.0
  flutter_shaders: ^0.1.3

  # State Management
  provider: ^6.1.1

  # Internationalisation
  intl: ^0.19.0

  # 3D & Media
  media_kit: ^1.1.10+1
  media_kit_video: ^1.2.4
  media_kit_libs_windows_video: ^1.0.9
```

```
  media_kit_libs_android_video: ^1.0.8
  flutter_cube: ^0.1.1
  o3d: ^3.1.3

  # WebView
  webview_flutter: ^4.4.1
  webview_windows: 0.4.0
  webview_win_floating: ^3.0.2

  # File Management
  file_picker: ^10.3.10

  # Firebase
  firebase_core: ^4.5.0
  firebase_auth: ^6.2.0
  firebase_app_check: ^0.4.1+5
  cloud_firestore: ^6.1.3
  firebase_storage: ^13.1.0

  # Local Storage
  sqflite: ^2.4.1
  path: ^1.9.1
  shared_preferences: ^2.3.5
  sqflite_common_ffi: ^2.3.3

  # Media Recording & Playback
  record: ^6.2.0
  audioplayers: ^6.1.0
  path_provider: ^2.1.2
  image_picker: ^1.1.2

  # Device
  device_info_plus: ^10.1.0

  # AI
  google_generative_ai: ^0.4.7

  # Utilities
  equatable: ^2.0.7
  logging: ^1.3.0
  motor: ^1.0.1
```

## Appendix B — Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read: if true;
      allow write: if request.auth != null
                   && request.auth.uid == userId;
    }
    match /chats/{chatId} {
      allow read, write: if request.auth != null;
    }
```

```
    match /messages/{messageId} {
      allow read, write: if request.auth != null;
    }
    match /stories/{storyId} {
      allow read:   if request.auth != null;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
                            && request.auth.uid == resource.data.userId;
    }
    match /contacts/{contactId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

## Appendix C — Hardware and Software Requirements Summary

### *C.1 End-User Device Requirements*

| Requirement | Minimum | Recommended |
|---|---|---|
| Platform | Android 6.0 (API 23) | Android 10+ (API 29+) |
| RAM | 2 GB | 4 GB |
| Free Storage | 100 MB | 500 MB |
| Network | Mobile data or Wi-Fi | Wi-Fi / 4G / 5G |
| Camera | Optional (avatar, stories) | Required for full features |
| Microphone | Optional (voice messages) | Required for audio features |
| Screen | 720 × 1280 px | 1080 × 1920 px or higher |

### *C.2 Development Machine Requirements*

| Requirement | Specification |
|---|---|
| OS | Windows 10/11, macOS 12+, or Ubuntu 20.04+ |
| RAM | 8 GB minimum (16 GB recommended) |
| Storage | 10 GB free (Flutter SDK + Android SDK + project) |
| Flutter SDK | 3.0.0 or higher |
| Dart SDK | 3.0.0 or higher |
| Android Studio | Hedgehog 2023.1 or newer |
| Java JDK | Version 17 |
| Git | 2.x |
| Internet | Required for Firebase, Gemini API, and Pub package downloads |

### *C.3 Firebase Project Requirements*

| Service | Plan Required | Purpose |
|---|---|---|
| Firebase Authentication | Free (Spark) | User login and registration |
| Cloud Firestore | Free (Spark) — up to 1 GB | Real-time data storage |
| Firebase Storage | Free (Spark) — up to 5 GB | Media file storage |
| Firebase App Check | Free | Request verification |

### *C.4 Third-Party API Requirements*

| API | Provider | Purpose |
|---|---|---|
| Gemini 2.0 Flash | Google AI Studio | AI chatbot and sentiment analysis |
| Google Fonts | Google Fonts CDN | Typography |

End of Document